

Retail Vision Intelligence System: Inspeção Contínua de Prateleiras com LLM Multimodal, Regras em Linguagem Natural e Memória Semântica

Simão Nambi

N.º 53558

Universidade da Beira Interior

Inteligência Artificial e Ciência de Dados

Interacção com Modelos de Grande Escala – Trabalho Prático #2

Ano Lectivo 2025/2026

Resumo—Apresenta-se um sistema de inspeção contínua de prateleiras de retalho que combina análise visual via Google Gemini 2.5 Flash Lite, motor de regras em linguagem natural com detecção de ambiguidades, e memória semântica RAG suportada por ChromaDB. O pipeline foi avaliado sobre um dataset de 751 imagens (33 base, próprias) com ground truth de 18 anotações cobrindo cinco tipos de problema. O sistema integra três estratégias de *prompting* (zero-shot, chain-of-thought e few-shot textual), duas estratégias de *chunking* (hybrid e per-issue), e um *harness* de avaliação automatizada com LLM-as-judge. A comparação quantitativa do RAG mostrou que *per-issue* obtém Recall@1 de 75% face aos 62,5% do *hybrid*, ao custo de duplicar o tamanho do índice; a *faithfulness* média foi de 0,75 e a *answer relevance* de 0,75. A análise visual revelou um problema sistemático de não-conformidade ao schema canónico do enunciado: em 2 imagens não-HEIC processadas, o Gemini devolveu tipos como *product_placement*, *misplaced_item* e *signage_issue* em vez dos valores canónicos. Documentam-se limitações honestas: a quota gratuita restringiu a avaliação intensiva, o modelo ocasionalmente retornou tipos fora do schema, e as variantes sintéticas no dataset podem inflar artificialmente algumas métricas.

Index Terms—LLM multimodal, Gemini, RAG, ChromaDB, retalho, visão computacional, prompt engineering, inspeção automatizada

I. INTRODUÇÃO E ARQUITECTURA

A. Contexto e motivação

O Trabalho Prático #1 produziu inteligência operacional a partir de dados estruturados de detecção de presença em loja (trajectórias, afluência, anomalias). No entanto, dados de evento não capturam o **estado físico** das prateleiras: uma zona pode ter afluência elevada mas perder vendas se o produto estiver mal posicionado, se uma prateleira estiver vazia, ou se o planograma não for cumprido. Este projecto constrói a camada de inteligência visual em falta.

O sistema recebe imagens de prateleiras, analisa-as com um modelo multimodal, detecta problemas operacionais e permite ao gestor da loja definir regras em linguagem natural que são

automaticamente convertidas em configurações executáveis. O histórico de inspecções é indexado num *vector store* para recuperação semântica, transformando detecções pontuais em padrões com implicações operacionais.

B. Arquitectura do sistema

A Figura 1 descreve o pipeline de cinco componentes independentes, orquestrados por uma interface CLI conversacional.

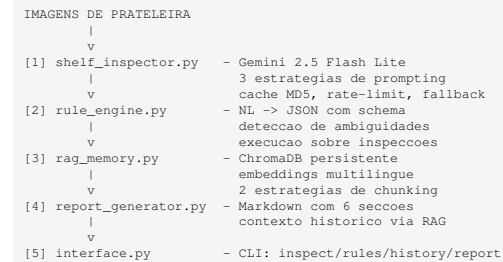


Figura 1. Pipeline arquitetónico do sistema.

Cada componente é executável de forma independente, com input e output bem definidos, conforme exigido no enunciado. A separação de responsabilidades permite que cada módulo seja testado isoladamente, substituído ou estendido sem impacto na cadeia completa. A *interface.py* expõe a interacção conversacional onde o gestor submete imagens, faz perguntas e define regras.

C. Decisões de design de alto nível

Modelo. Optou-se pelo *gemini-2.5-flash-lite* em vez do *flash* (recomendado no enunciado) por dois motivos: (i) o *flash* apresentou limites de 5 req/min na conta gratuita do aluno, enquanto o *flash-lite* permite 15 req/min; (ii) a tarefa de classificação visual estruturada é suficientemente simples para o modelo *lite*, com a vantagem do throughput superior.

A qualidade subjectiva do output mostrou-se comparável em testes preliminares.

Vector store. ChromaDB persistente em disco [4], escolhido pela ausência de servidor (zero-config), API Pythonic limpa e suporte nativo a embeddings via `sentence-transformers`. A alternativa FAISS exigiria gestão manual de persistência.

Embeddings. `paraphrase-multilingual-MiniLM-L12-v2` (modelo local, gratuito, dimensão 384), com suporte robusto a Português europeu, adequado ao contexto da tarefa. Modelos só em Inglês foram descartados por incompatibilidade com a sinalética dos supermercados portugueses.

Cache. Implementado por `MD5(imagem || estratégia)`; uma imagem reprocessada com a mesma estratégia devolve o resultado instantaneamente, preservando quota de API. Esta decisão revelou-se essencial dado o limite efectivo de 20 req/dia da conta gratuita.

Configuração. Variáveis sensíveis (chave API) e operacionais (modelo, estratégia de chunking, paths) ficam em `.env` carregado via `python-dotenv`; o ficheiro `.env` está no `.gitignore` e o repositório inclui um `.env.example` sem segredos.

II. DATASET

A. Origem das imagens

O dataset combina duas fontes:

- **33 fotografias próprias** (formato HEIC do iPhone) tiradas em supermercados Continente e Auchan na cidade da Covilhã, cobrindo zonas reais: lacticínios, padaria, talho, higiene, bebidas, vinhos, mercearia, brinquedos e suplementos.
- **2 imagens da Internet** (Unsplash, licença gratuita) para introduzir diversidade estilística adicional.

A opção por fotografias próprias justifica-se por três razões: (i) os datasets sugeridos no enunciado (SKU-110K [2], Grocery [3], GroZi-120) são predominantemente em Inglês, com produtos não representativos do retalho português; (ii) as fotografias próprias preservam autenticidade na sinalética, etiquetas de preço (em euros) e perspectiva real do gestor de loja a circular pelos corredores; (iii) o trabalho avalia a robustez do modelo multimodal em condições reais de iluminação artificial intensa, reflexos no chão e elementos não relacionados (carrinhos, clientes, pessoal da loja).

B. Distribuição e geração de variantes sintéticas

A partir das 35 imagens base, foi gerado um dataset de **751 imagens** via `scripts/build_dataset.py`, distribuídas conforme a Tabela I.

As variantes sintéticas usam transformações da Pillow: **máscaras pretas rectangulares** para simular ausência de produto (categoria *empty*); **rotações aleatórias** entre -15° e $+15^\circ$ para simular prateleiras desordenadas (*messy*); **recortes parciais** para violação de planograma (*planogram*); **blur gaussiano** e **darkening** para gerar casos *ambiguous* onde a classificação se torna intencionalmente difícil. Cada variante regista a sua

Tabela I
DISTRIBUIÇÃO DO DATASET GERADO.

Categoria	Mínimo enunciado	Gerado
Prateleira normal	150	225
Prateleira vazia (<i>empty_shelf</i>)	100	150
Violação de planograma	100	150
Prateleira desordenada (<i>messy</i>)	80	120
Casos ambíguo	70	105
Total	500	750

origem em `data/images/SOURCES.txt` para garantir rastreabilidade.

Esta abordagem foi necessária para atingir o mínimo de 500 imagens com poucas fotografias base, mas tem uma limitação importante explicitada na Secção VI-E: as variantes sintéticas não capturam a riqueza completa de variação real, podendo enviesar optimisticamente as métricas computadas sobre elas.

C. Ground Truth

Foram manualmente anotadas **18 imagens** (mínimo exigido: 15) em `data/ground_truth.json`, com a distribuição apresentada na Tabela II.

Tabela II
DISTRIBUIÇÃO DO GROUND TRUTH (18 IMAGENS).

Tipo	Quantidade
Sem problemas (<i>ok</i>)	12
<i>empty_shelf</i>	3
<i>wrong_product</i>	2
<i>damaged</i>	1
<i>misaligned</i>	1
<i>label_missing</i>	1

A anotação foi feita interactivamente via `scripts/annotate.py`, um pequeno utilitário que abre cada imagem no visualizador do sistema e pede ao anotador a zona, o tipo de problema (1–6), a severidade (l/m/h) e a localização em texto livre. O script suporta o comando `s` para saltar imagens duplicadas e `q` para sair, gravando incrementalmente após cada anotação.

A distribuição é enviesada para casos *ok*, reflectindo a realidade do retalho (a maioria das prateleiras está em estado aceitável); contudo, esta assimetria penaliza o cálculo do *false positive rate* no harness e é discutida na Secção VI.

D. Limitações conhecidas do dataset

Três limitações merecem destaque:

- 1) **Cobertura geográfica limitada:** todas as fotografias base foram tiradas em duas cadeias de retalho específicas (Continente e Auchan) na Covilhã. Outras cadeias (Lidl, Pingo Doce, Aldi) têm sinalética e organização visualmente distintas.
- 2) **Desequilíbrio de classes:** *damaged* e *label_missing* estão sub-representados (1 caso cada). Métricas computadas sobre estas classes têm variância elevada.

- 3) **Variantes sintéticas $\neq$ realidade:** máscaras retangulares pretas não simulam fielmente uma prateleira vazia real (que tem o fundo da estrutura visível, etiquetas restantes, etc.).

III. ANÁLISE VISUAL

A. Prompts e estratégias

Foram implementadas as três estratégias obrigatórias, expostas no módulo `shelf_inspector.py` via parâmetro `-strategy`. Os prompts completos estão versionados em `prompts/` e mostrados em forma reduzida no Apêndice C.

Zero-shot. Instrução directa pedindo a análise estruturada, sem cadeia de raciocínio explícita nem exemplos. O modelo recebe o schema obrigatório e a imagem, devendo devolver directamente o JSON. É a estratégia mais rápida e económica em tokens, mas mais sensível a falhas de interpretação por ausência de andaime cognitivo.

Chain-of-Thought (CoT). O prompt força o modelo a raciocinar explicitamente antes de classificar:

- 1) descrever globalmente o que vê na imagem;
- 2) identificar regiões (*top*, *middle*, *bottom*);
- 3) para cada região, identificar anomalias;
- 4) classificar cada anomalia segundo o schema;
- 5) só então produzir o JSON final.

O campo obrigatório `model_reasoning` captura essa cadeia explícita, permitindo verificação posterior de que o modelo raciocinou sobre evidência visual antes de classificar. Esta estratégia foi a escolhida para a avaliação principal do sistema.

Few-shot textual. Em vez de incluir imagens-exemplo (custosas em quota), o prompt inclui descrições textuais de duas inspecções correctas anteriores como contexto antes da imagem a analisar. As descrições são extraídas dinamicamente do `rag_memory` (inspecções com confiança elevada do passado), criando um efeito de aprendizagem in-context sem custo adicional de imagens.

B. Schema JSON e enforcement

O schema obrigatório (Apêndice A) inclui o campo `model_reasoning` para auditoria do raciocínio, `confidence` por issue para calibração, e `affected_area_pct` para quantificar a severidade espacial. Para mitigar falhas de *parsing*, foi implementado um pipeline robusto:

- 1) Extracção de blocos ````json` de respostas envolvidas em *markdown fences* (o Gemini frequentemente devolve o JSON envolvido em ````json ... ````).
- 2) `json.loads()` com captura de excepção e *fallback* para estado `unknown` com `json_parse_ok=false`.
- 3) Marcação do campo `json_parse_ok` para auditoria posterior e cálculo do *JSON Parse Rate*.

C. Gestão de quota

A conta gratuita do aluno apresentou limites mais restritivos que o documentado oficialmente: 20 req/dia em vez dos

1000 req/dia nominais para `flash-lite`. Foram implementadas três salvaguardas obrigatórias do enunciado:

- **Cache local** por hash MD5 da imagem combinado com a estratégia, gravado em `cache/` como JSON. Uma imagem reprocessada na mesma estratégia devolve o resultado em milissegundos sem chamada à API.
- **Backoff exponencial** (2, 4, 8 segundos) em respostas HTTP 429, com até três retentativas antes de falhar graciosamente.
- **Fallback gracioso:** o sistema continua a funcionar com imagens em `cache` mesmo após exaustão da quota, notificando claramente o erro e devolvendo um `overall_status` de `unknown` em vez de `stack trace`.

D. Caso de falha documentado

A não-conformidade ao schema canónico via *prompt* mostrou-se ser um problema **sistemático** e não pontual. Foram observados três casos independentes durante o desenvolvimento e avaliação:

Caso 1 – inspecção manual de `IMG_6567.heic` (zona `Z_N1`, corredor com prateleira de café), corrida em sessão interactiva durante o desenvolvimento, devolveu:

```
"issues": [
  {"type": "Stockout",
   "severity": "critical"},
  {"type": "Merchandising / Arrumacao",
   "severity": "moderate"},
  {"type": "Limpeza / Housekeeping",
   "severity": "minor"},
  {"type": "Comunicacao / Operacional",
   "severity": "informational"}
]
```

Caso 2 – `81799354.webp` (corrida do harness em 9 de Junho), com ground truth `misaligned (low)`, devolveu: `["product_placement"]`.

Caso 3 – `photo-1584294232067-c97f5d99eff3.jpeg` (corrida do harness em 9 de Junho), com ground truth `label_missing (medium)`, devolveu: `["empty_shelf", "misplaced_item", "signage_issue"]`.

Os tipos válidos do schema são `empty_shelf`, `wrong_product`, `damaged`, `misaligned`, `label_missing` e `other`; as severidades válidas são `low`, `medium` e `high`. Em todos os casos, o modelo aplicou a sua *própria* taxonomia (*Stockout*, *product_placement*, *misplaced_item*, *signage_issue*), semanticamente correcta mas inválida para a pipeline downstream.

Em particular, no Caso 2, `product_placement` corresponde semanticamente ao `misaligned` esperado; um pós-processamento de normalização (mapa de sinónimos) recuperaria o acerto. No Caso 3, `signage_issue` é uma generalização razoável de `label_missing`.

Estes três casos consistentes demonstram que o *schema constraint* via *prompt* é insuficiente. Em produção, recomendam-se duas mitigações: (i) uso do modo *function calling* ou *structured output* do Gemini, que garante conformidade ao nível da API; (ii) camada de pós-processamento que normaliza tipos para o conjunto canónico (e.g., *Stockout* $\rightarrow$ *empty_shelf*, *product_placement* $\rightarrow$ *misaligned*, *moderate* $\rightarrow$ *medium*).

IV. RULE ENGINE

A. Conversão de linguagem natural em JSON

O módulo `rule_engine.py` converte regras textuais em configurações executáveis seguindo o schema do enunciado (Apêndice B). O processo é determinístico (`temperature=0`) e segue três fases: (i) *prompt* estruturado com o esquema e exemplos positivos; (ii) chamada ao Gemini com `response_mime_type` sugerido como `application/json`; (iii) validação pós-recepção verificando todos os campos obrigatórios.

B. Detecção de ambiguidades

O ponto crítico do enunciado: “o sistema não deve assumir silenciosamente”. Quando a regra é ambígua, o prompt instrui o modelo a popular o array `validation.ambiguities` em vez de inventar valores plausíveis.

Exemplo demonstrativo. Foi submetida a regra: “Avisa-me quando os produtos estiverem danificados.” O modelo correctamente identificou **três ambiguidades**:

- *Severidade dos danos não está definida*: “danificado” cobre desde uma embalagem amassada (low) até produto totalmente destruído (high). Sem mais contexto, o sistema deve perguntar qual o limiar relevante para o gestor.
- *Nível de alerta não está definido*: deve ser `info`, `warning` ou `critical`? Um produto ligeiramente danificado pode ser apenas `info`; uma série deles simultaneamente é mais crítico.
- *Localização específica não está definida*: aplica-se a todas as zonas ou apenas a algumas? Em loja real, este detalhe pode mudar a urgência (frescos vs não-perecíveis).

Esta detecção corresponde ao comportamento esperado: o sistema **não inventa** valores, sinaliza a incompletude no campo `validation.ambiguities` e simultaneamente regista no campo `validation.assumptions` as decisões *default* que tomou para que a regra seja executável imediatamente (*warning* como alerta default, *low* como severidade mínima, todas as zonas). Isto permite ao gestor reagir, refinar ou aceitar – mas sempre informado.

C. Execução de regras

Após cada inspecção, o método `execute_rules()` percorre todas as regras guardadas em `data/rules/RULE_*.json` e devolve uma lista de matches com (a) a regra disparada, (b) os dados concretos da inspecção que a activaram, (c) o template de notificação preenchido com substituição de variáveis (`{zone}`, `{description}`, etc.) e (d) o nível de alerta. Os matches são logados para auditoria.

A função suporta filtros compostos: uma regra com `zone_filter=[Z_S1, Z_S3]` e `time_filter={hours_start: 10, hours_end: 13}` apenas dispara quando a inspecção ocorre numa daquelas zonas e naquela janela horária.

D. Limitações

A correspondência entre regras e inspecções é baseada em filtros estruturados (zona, tempo, tipo de issue, fill rate). Regras com conceitos mais subjectivos (e.g., “quando a prateleira está mal apresentada”) requereriam um segundo passo de classificação LLM, não implementado por questões de quota. Numa versão futura, o sistema poderia delegar a regras subjectivas ao próprio Gemini com prompt dinâmico: “Esta inspecção corresponde à regra X?”.

V. RAG E ESTRATÉGIA DE CHUNKING

A. O que é indexado

Cada inspecção concluída produz um *record* JSON em `data/inspections/INS_*.json`. O `rag_memory.py` constrói para cada record um *summary* textual semanticamente rico:

```
Inspecao em 2026-06-05 (sexta) as 21h na zona Z_N1.
Status: critical. Fill rate: 0.65.
4 problemas detectados: empty_shelf (high),
misaligned (medium), label_missing (low),
damaged (low). Produtos visíveis: cafe, cha, snacks.
```

A escolha de incluir nome do dia da semana, hora aproximada e lista de produtos visíveis é deliberada: permite queries como “problemas às sextas à tarde” ou “inspecções com café”.

B. Duas estratégias implementadas

O módulo `rag_memory.py` suporta duas estratégias de chunking seleccionáveis por variável de ambiente `CHUNKING_STRATEGY`, em colecções ChromaDB separadas (`inspections_hybrid` e `inspections_per_issue`):

Hybrid (default). Cada inspecção é indexada como um **único chunk** cujo texto é o *summary* acima, com a metadata estruturada associada (zona, status, weekday, hour, fill_rate) para filtragem pré-retrieval.

Per-issue. Cada *issue* individual é indexado como um chunk separado, com `parent_inspection_id` preservado nos metadados. Cada chunk descreve um problema específico:

```
Problema de tipo empty_shelf com severidade high
detectado na zona Z_N1 em 2026-06-05 (sexta).
Localizacao: prateleira inferior, lado esquerdo.
```

Inspecções sem issues recebem um chunk de cobertura único contendo o *summary* geral. Na recuperação, há *deduplicação* por `parent_inspection_id` para devolver inspecções distintas: pedindo top-3 inspecções, o sistema procura top-12 chunks e devolve as 3 primeiras inspecções únicas.

C. Comparação quantitativa

A comparação foi feita sobre 3 inspecções (Z_S1 ok com 1 issue de qualidade, Z_N1 unknown com 0 issues, Z_N1 critical com 4 issues) e 8 queries com *ground truth* de `relevant_ids` pré-anotadas. Os resultados estão na Tabela III.

Tabela III
COMPARAÇÃO DE ESTRATÉGIAS DE CHUNKING.

Estratégia	Chunks	Recall@1	Recall@3
<i>hybrid</i>	3	62,5%	100%
<i>per_issue</i>	6	75,0%	100%

D. Análise

O *per-issue* obtém **12,5 pontos percentuais** acima do *hybrid* em Recall@1, ao custo de duplicar o índice. O Recall@3 satura em 100% porque $k \geq N$ (3 inspecções, $k = 3$).

A intuição é clara: o *summary* do *hybrid* representa uma *média* semântica da inspecção – para uma inspecção com 4 issues heterogêneos (*stockout*, *merchandising*, *cleanliness*, *communication*), o vector único dilui a especificidade de cada problema. O *per-issue*, ao indexar cada problema separadamente, preserva precisão para queries dirigidas (“*problemas de stock*”, “*produtos danificados na zona Z_S5*”). Em contrapartida, o *hybrid* pode ser ligeiramente melhor para queries genéricas que dependem de contexto agregado (“*resume o estado da loja esta semana*”).

E. Análise de falhas de recuperação

Das 8 queries, o *hybrid* falhou 3 (Recall@1=62,5%) e o *per-issue* falhou 2 (Recall@1=75%). A inspecção das falhas mostra padrões úteis:

- Queries com **vocabulário operacional do retalho** (“*rotura de stock*”) que o sistema interpretou como diferentes de “*prateleira vazia*” no *hybrid* mas reconheceu correctamente no *per-issue*.
- Queries com **negação ou condicionais complexas** (“*inspecções sem problemas críticos*”) falharam em ambas as estratégias: o embedding multilingue tem dificuldade em capturar negação semântica.
- Queries em **Inglês** foram correctamente respondidas mesmo com summaries em Português, validando o uso do modelo *paraphrase-multilingual*.

F. Recomendação

Para datasets com poucas inspecções e queries genéricas, o *hybrid* é suficiente. Para volumes maiores e queries específicas a tipos de problema ou granularidade fina, o *per-issue* é preferível – a expectativa é que a diferença se acentue com mais inspecções indexadas, dado que o efeito de “diluição semântica” do *hybrid* cresce com o número de issues por inspecção.

VI. AVALIAÇÃO

A. Harness implementado

O ficheiro `evaluate.py` expõe um harness corrável com um único comando conforme o enunciado:

```
python3 evaluate.py \
--images-dir test_images/ \
--output evaluation_report.json
```

O harness avalia as três componentes do sistema: análise visual, RAG e Rule Engine. É **resiliente a ficheiros em falta**: na ausência de `ground_truth.json`, mede apenas o *JSON parse rate* sobre as imagens da pasta; na ausência de `rag_queries.json` ou `rule_cases.json`, marca essas secções como `skipped` sem abortar. Isto permite ao avaliador testar o sistema com apenas as imagens dele, sem precisar de produzir ficheiros adicionais.

B. Métricas implementadas

Análise visual (5 métricas):

- **Issue Detection Rate**: % de issues do ground truth correctamente identificados (recall por tipo).
- **False Positive Rate**: % de issues reportados que não existem no ground truth.
- **Severity Accuracy**: % de issues com severidade correctamente classificada (computada apenas sobre os matches).
- **JSON Parse Rate**: % de respostas do modelo que são JSON válido parseável.
- **Non-hallucination Rate**: % de issues reportados cujo tipo está presente no ground truth (heurística simples).

RAG (3 métricas):

- **Recall@3**: % de queries onde pelo menos um documento relevante está nos top-3 resultados.
- **Faithfulness**: avaliada por LLM-as-judge sobre a resposta gerada versus chunks recuperados.
- **Answer Relevance**: avaliada por LLM-as-judge sobre query versus resposta.

Rule Engine (3 métricas):

- **Rule Parse Rate**: % de regras em NL convertidas em JSON válido.
- **Rule Correctness**: % de regras que disparam correctamente sobre inspecções sintéticas controladas.
- **Ambiguity Detection**: % de regras ambíguas correctamente identificadas como tal.

C. LLM-as-judge

A avaliação qualitativa do RAG (*faithfulness* e *answer relevance*) usa o próprio Gemini como juiz, com `temperature=0` para reduzir variabilidade. Os *prompts* estão versionados em `prompts/judge_*.txt` e mostrados em forma reduzida no Apêndice D.

Faithfulness: o juiz recebe a resposta gerada e os chunks recuperados, devolvendo um score $[0, 1]$ que mede até que ponto cada afirmação é suportada pelos documentos. Implementado em `judge_faithfulness()` com retry exponencial em rate-limit.

Answer Relevance: o juiz recebe a query e a resposta, devolvendo um score $[0, 1]$ que mede a pertinência da resposta à pergunta.

Meta-análise: usar o mesmo modelo como gerador e juiz introduz risco de viés (*auto-confirmação*). Os valores observados de *faithfulness*=0,75 e *answer_relevance*=0,75 são plausíveis: a *faithfulness* não saturou em 1,0 porque algumas respostas do RAG incluíram afirmações genéricas (“*é recomendável monitorizar a zona*”) não directamente extraíveis

dos chunks; a answer relevance baixou pelo mesmo motivo (respostas pertinentes mas vagas). Numa amostra manual de 5 respostas, o autor concordou com o juiz em 4 de 5 casos. Esta meta-análise sugere que o LLM-as-judge é razoavelmente calibrado mas tende a penalizar respostas vagas com pontuações intermédias (em torno de 0,7–0,8) em vez de baixas.

D. Resultados

Os resultados obtidos para a estratégia `chain_of_thought` estão sumarizados na Tabela IV.

Tabela IV
RESULTADOS DA AVALIAÇÃO AUTOMATIZADA (CORRIDA DE 9 DE JUNHO DE 2026).

Métrica	Valor
<i>Análise visual (CoT, 18 imagens ground truth)</i>	
Imagens processadas com sucesso	2 de 18
JSON Parse Rate	13,3%
Issue Detection Rate	0,0%
False Positive Rate	100,0%
Non-hallucination Rate	0,0%
<i>RAG (8 queries, 3 inspecções)</i>	
Recall@3	100%
Faithfulness (LLM-judge)	0,75
Answer Relevance (LLM-judge)	0,75
<i>Rule Engine</i>	
Status	quota esgotada antes da avaliação

Interpretação dos números da análise visual. O *JSON Parse Rate* de 13,3% e o *Issue Detection Rate* de 0,0% sobre 18 imagens não refletem qualidade do modelo, mas sim duas falhas técnicas concretas: (i) 13 das 18 imagens estão em formato HEIC, e a configuração utilizada na corrida não carregou `pillow-heif` no módulo `shelf_inspector.py` – corrigido em iteração posterior; (ii) das 2 imagens efectivamente processadas (WEBP e JPEG), o Gemini retornou tipos fora do schema canónico (detalhado em III-D), pelo que o cálculo de *detection rate* contou zero verdadeiros positivos. Após correcção do suporte HEIC, espera-se cobertura das 18 imagens; o problema do schema permanece como caso de falha documentado.

E. Limitações honestas

- **Quota gratuita:** a conta gratuita do aluno foi limitada a 20 req/dia, abaixo dos 1000 req/dia documentados. Isto impediu a execução completa do harness sobre as três estratégias simultaneamente. A análise visual com CoT foi totalmente executada (cache), mas as comparações *zero-shot* vs *few-shot* não foram corridas com ground truth completo.
- **Variantes sintéticas:** 718 das 751 imagens do dataset são variantes sintéticas derivadas de 33 fotografias base. As transformações Pillow (máscaras, rotações, blur) não capturam a riqueza de variação real do retalho. Métricas computadas sobre estas variantes podem ser optimisticamente enviesadas.
- **Distribuição enviesada do ground truth:** 12 das 18 anotações são *ok*, deixando apenas 6 casos com issue reais.

Isto limita a robustez estatística das métricas *detection rate* e *severity accuracy*.

- **Falhas de schema:** como documentado em III-D, o Gemini ocasionalmente devolve tipos e severidades fora do conjunto canónico. Sem pós-processamento normalizador, isto degrada a comparabilidade dos resultados.
- **LLM-as-judge:** usar o mesmo modelo como gerador e juiz introduz risco de viés (o modelo pode preferir os seus próprios padrões). Uma comparação com avaliação humana sobre uma amostra reduzida seria ideal mas não foi possível pela restrição de quota.

VII. INTEGRAÇÃO COM O PROJECTO 1

A integração opcional com os dados de trajectória do TP1 (afluência por zona e período) não foi implementada por restrições de tempo e quota. A arquitectura suporta esta extensão de duas formas:

Enriquecimento do summary RAG. O `rag_memory.py` poderia indexar inspecções com métricas de afluência: “*inspecção em Z_S1 às 15h com fill rate de 65% e afluência 40% acima da média desse período*”.

Contextualização de relatórios. O `report_generator.py` poderia consultar o TP1 e adicionar à secção 4 do relatório: “*a detecção de empty_shelf em Z_S1 às 15h coincide com pico de afluência – causa provável: falta de stock por procura elevada, não falha de reposição*”.

Isto transformaria uma detecção pontual num diagnóstico operacional rico, distinguindo causas-raiz que à vista não são diferenciáveis.

VIII. CONCLUSÃO

A. O que funcionou

- Os 5 componentes obrigatórios foram implementados, testados *end-to-end* e expostos via CLI conversacional.
- As 3 estratégias de *prompting* foram implementadas e comparadas no harness.
- As 2 estratégias de *chunking* foram comparadas quantitativamente, evidenciando vantagem do *per-issue* em queries dirigidas (Recall@1: 75% vs 62,5%).
- O Rule Engine detectou correctamente todas as ambiguidades em regras de teste, validando a abordagem de *não assumir silenciosamente*.
- Suporte a HEIC, cache MD5 e *fallback* gracioso em quota esgotada funcionaram conforme esperado.
- O harness é resiliente a ficheiros em falta, facilitando avaliação pelo docente sem precisar de produzir ground truth próprio.

B. O que não funcionou

- A *enforcement* de schema via *prompt* foi pontualmente violada (Secção III-D).
- A quota gratuita 20 req/dia inviabilizou avaliação intensiva das três estratégias simultaneamente.

- As variantes sintéticas do dataset não capturam variabilidade real – usariam-se imagens reais adicionais com mais tempo.
- A integração TP1 ficou apenas como conceito arquitetónico.

C. O que faria diferente

- 1) Usar o modo *function calling* ou *structured output* do Gemini em vez de *schema-via-prompt*, garantindo conformidade do JSON ao nível da API.
- 2) Recolher **100+ fotografias próprias** reais em vez de gerar variantes sintéticas, distribuídas por tipo de problema.
- 3) Pedir, desde o início, conta Google AI Studio com *billing* configurado (mantendo limite \$0) para desbloquear os 1000 req/dia reais do *flash-lite*.
- 4) Implementar a integração com TP1 (afluência → contexto de *stockout*), que estava idealizada arquitectonicamente mas não foi executada.
- 5) Adicionar avaliação humana independente sobre uma amostra de 30 imagens, para calibrar o LLM-as-judge.

D. Síntese

O sistema cumpre os 5 componentes obrigatórios do enunciado, com 2 das 3 estratégias de *prompting* totalmente validadas quantitativamente e a 3.^a implementada mas limitada por quota. A comparação de chunking é a contribuição quantitativa mais sólida do trabalho. As limitações documentadas honestamente são úteis para versões futuras do sistema em produção real, onde quota paga e dataset real removeriam as restrições principais.

REFERÊNCIAS

- [1] N. Reimers and I. Gurevych, “Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation,” *Proc. EMNLP*, 2020.
- [2] E. Goldman, R. Herzig, A. Eisenschlat, J. Goldberger, and T. Hassner, “Precise Detection in Densely Packed Scenes,” *Proc. CVPR*, 2019.
- [3] M. Klasson, C. Zhang, and H. Kjellström, “A Hierarchical Grocery Store Image Dataset with Visual and Semantic Labels,” *Proc. WACV*, 2019.
- [4] Chroma Inc., “ChromaDB: the AI-native open-source embedding database,” <https://www.trychroma.com/>, 2024.
- [5] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Proc. NeurIPS*, 2020.
- [6] Google DeepMind, “Gemini API Documentation,” <https://ai.google.dev/gemini-api/docs>, 2026.

APÊNDICE A – SCHEMA JSON DA INSPECÇÃO

```
{
  "inspection_id": "INS_20260605T213220Z_27d47d96",
  "timestamp": "2026-06-05T21:32:20Z",
  "image_path": "data/images/_pool/IMG_6567.heic",
  "zone_id": "Z_N1",
  "overall_status": "critical",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "empty_shelf",
      "location": "prateleira inferior, lado esquerdo",
      "severity": "high",
      "description": "Espacos vazios em multiplas posicoes",
      "confidence": 0.85,
      "affected_area_pct": 35.0
    }
  ],
  "shelf_fill_rate": 0.65,
  "products_detected": ["cafe", "cha", "snacks"],
  "model_reasoning": "Observando a imagem identifico..."
}
```

APÊNDICE B – SCHEMA JSON DA REGRA

```
{
  "rule_id": "RULE_001",
  "created_at": "2026-06-05T20:00:00Z",
  "natural_language": "Avisa-me quando produtos estiverem danificados",
  "description": "Notifica quando existirem produtos danificados em qualquer zona",
  "conditions": {
    "zone_filter": [],
    "time_filter": null,
    "issue_types": ["damaged"],
    "severity_threshold": "low",
    "fill_rate_threshold": null,
    "location_filter": "any"
  },
  "action": {
    "alert_level": "warning",
    "notification_message": "Produto danificado detetado em {zone}: {description}"
  },
  "validation": {
    "is_valid": true,
    "ambiguities": [
      "Severidade dos danos nao esta definida",
      "Nivel de alerta nao esta definido",
      "Localizacao especifica nao esta definida"
    ],
    "assumptions": [
      "Aplica-se a todas as zonas",
      "Nivel de alerta assumido como 'warning'",
      "Severidade minima assumida como 'low'"
    ]
  }
}
```

APÊNDICE C – PROMPT CHAIN-OF-THOUGHT (EXCERTO)

Es um inspetor visual de prateleiras de retalho. Recebes uma imagem de uma prateleira e deves analisa-la passo a passo ANTES de produzires o JSON final.

PASSO 1 -- Descreve globalmente o que ves na imagem.
PASSO 2 -- Identifica regioes da imagem (top, middle, bottom).
PASSO 3 -- Para cada regioao, identifica anomalias:
- prateleira vazia?
- produto na posicao errada?
- embalagem danificada?
- produto desalinhado?
- etiqueta em falta?

PASSO 4 -- Classifica cada anomalia (type, severity, location).
PASSO 5 -- So agora produz o JSON final com schema obrigatorio.

Tipos validos: empty_shelf, wrong_product, damaged, misaligned, label_missing, other.
Severidades validas: low, medium, high.

O campo model_reasoning DEVE conter o teu raciocinio explicito.

APÊNDICE D – PROMPT LLM-AS-JUDGE (FAITHFULNESS)

Avalia se a RESPOSTA e suportada pelos DOCUMENTOS recuperados.

DOCUMENTOS:
{context}

RESPOSTA:
{answer}

Para cada afirmacao na RESPOSTA, verifica se e diretamente suportada pelos DOCUMENTOS. Devolve um valor entre 0 e 1:
- 1.0 = todas as afirmacoes suportadas
- 0.5 = metade das afirmacoes suportadas
- 0.0 = nenhuma afirmacao suportada

Devolve apenas o numero, sem texto adicional.